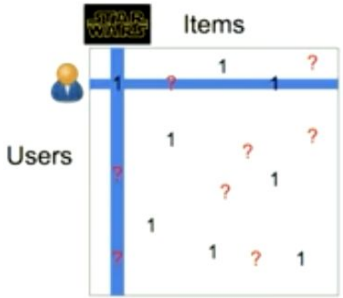
Reinforcement Learning for Recommender Systems: A Case Study on Youtube Video Recommendation

Minmin Chen (minminc@google.com)

Google

Refer: Video Paper

Gen1: Collaborative Filtering / Matrix Factorization

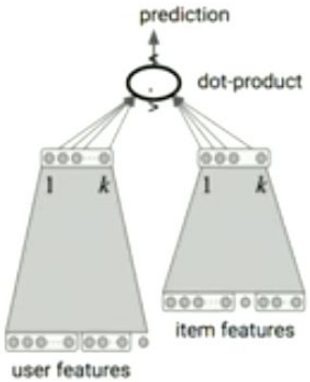


Given: Observed (user, item) interactions

Find: A model that predicts the missing interactions well.

$$\text{Loss}(\theta) = \sum_{(i,j)\in\mathcal{R}} (r_{i,j} - \mathcal{M}_\theta(i,j))^2$$

Gen 2: “Deeper” Learning



Richer modeling of users (DNNs, sequence models)

Richer modeling of items (DNNs)

Side information

1. System Bias:

Only Observe feedback on items recommended by current systems, but not others

?

- *Recommend immediate response instead of long term utility*
- *Can be eye catching but **hurt system in long term***

- Will depend on feedback from users
- Will know if the user likes these two videos
- *What about the videos which was not shown - but would have been liked by the user*

Will Trump Be '239 Pounds' At This Year's Physical?

short-term, familiar content

What Democrats' Tax Plans Could Mean For Growth And...

long-term, discovery content

"Pigeon-hole" users to recommendations that are likely to lead to immediate response instead of long term user utility

Where We Want To Be

Adapt and **discover** dynamic user preferences to optimize for long term user utility

Where we want to be

- **Adapt** and Discover Dynamic (**quickly**) User preferences
- to optimise for **long** term user utility

Challenges in Applying RL for Recommender Systems

- Large action space
- Expensive exploration
- Learning off-policy
- Partial observability
- Noisy reward

Extremely large action and state spaces:

- **action spaces** – many millions of **videos** to recommend
- **state** space - billions of **users**

Sparse data - Noisy Rewards:

- most users exposed to a small fraction of items
- explicit feedback provided to an even smaller fraction

Continuously-evolving user states:

- user preferences over these items are shifting all the time

Large action space: Deal with much bigger - millions/billions of videos

Exploration is expensive: - showing random movie - very bad user experience

Data is coming from off policy - Behaviour agent.

Learning off policy: But the behaviour agent has different policy than the policy we are learning

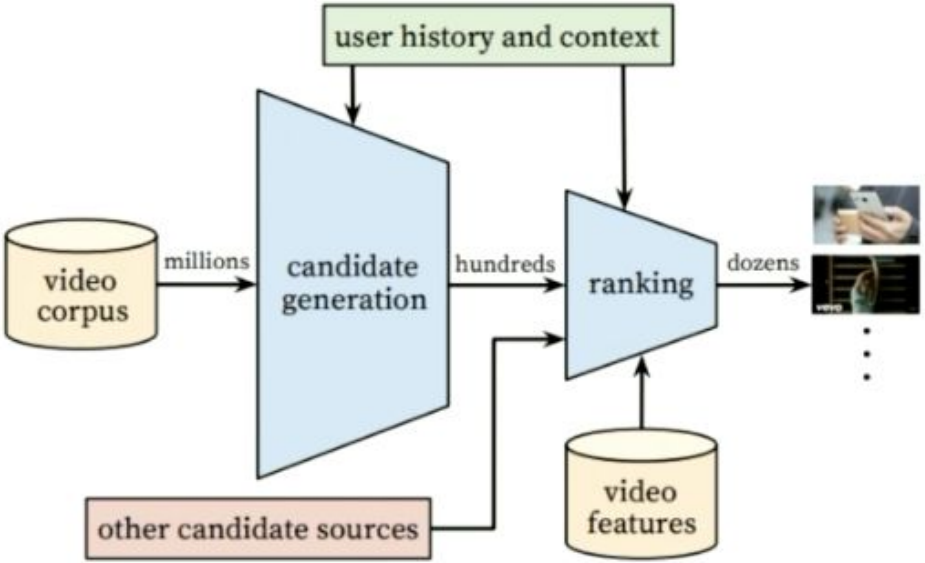
- So should be able to do effective off policy learning

Have noisy and sparse reward signal: Infer user interest from user activity - implicit feedback

- and sparse reward signals coming from users

A Case Study at Youtube: REINFORCE recommender

Youtube Recommendation Stack

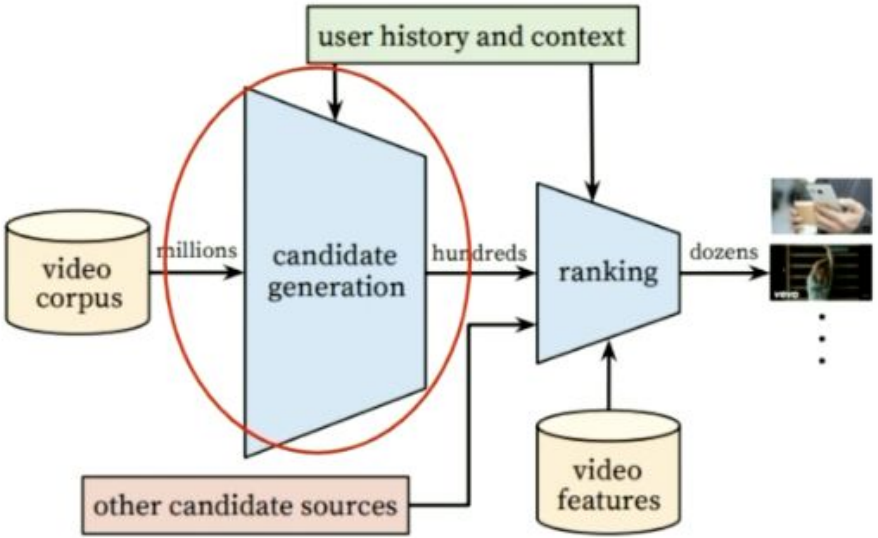


A multi-stage recommender system that selects dozens of videos to users from a corpus of billions of videos.

Covington, P., Adams, J., & Sargin, E.,. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (pp. 191-198). ACM.

Focus is on Candidate Generator

Candidate Generator



Billions of users with shifting preferences

Billions of items following long-tail distribution and dynamic

Noisy and sparse user feedback

Long tail: lot of videos are not receiving tons of views

- Still relevant - we want to recommend them
- Will have sparse and noisy feedback

RL for Recommender Systems

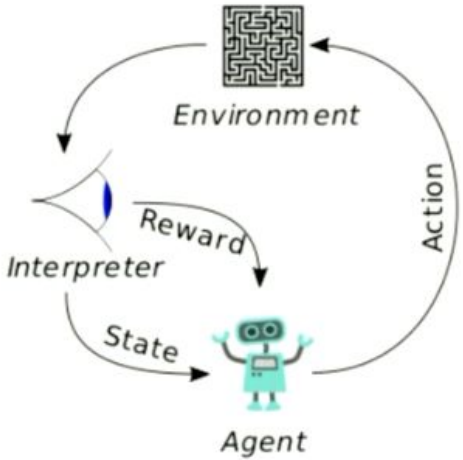
Building agents that take **actions** *in some environment* so as to maximize some notion of **cumulative reward**

Agent: candidate generator

State: user interest, context

Reward: user satisfaction

Action: nominate from a catalog of millions of videos



Some notion of Cumulative Reward

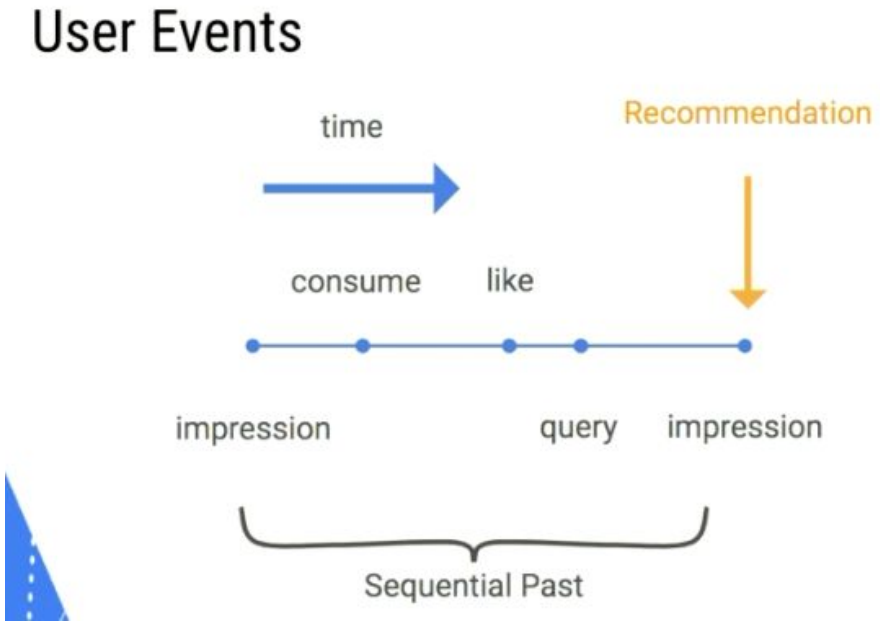
Environment is captured by: **state**, state **transition** and **reward** function

Reward: User's long term engagement and satisfaction with recommendation

Action by Agent - nominate from catalog of millions of videos

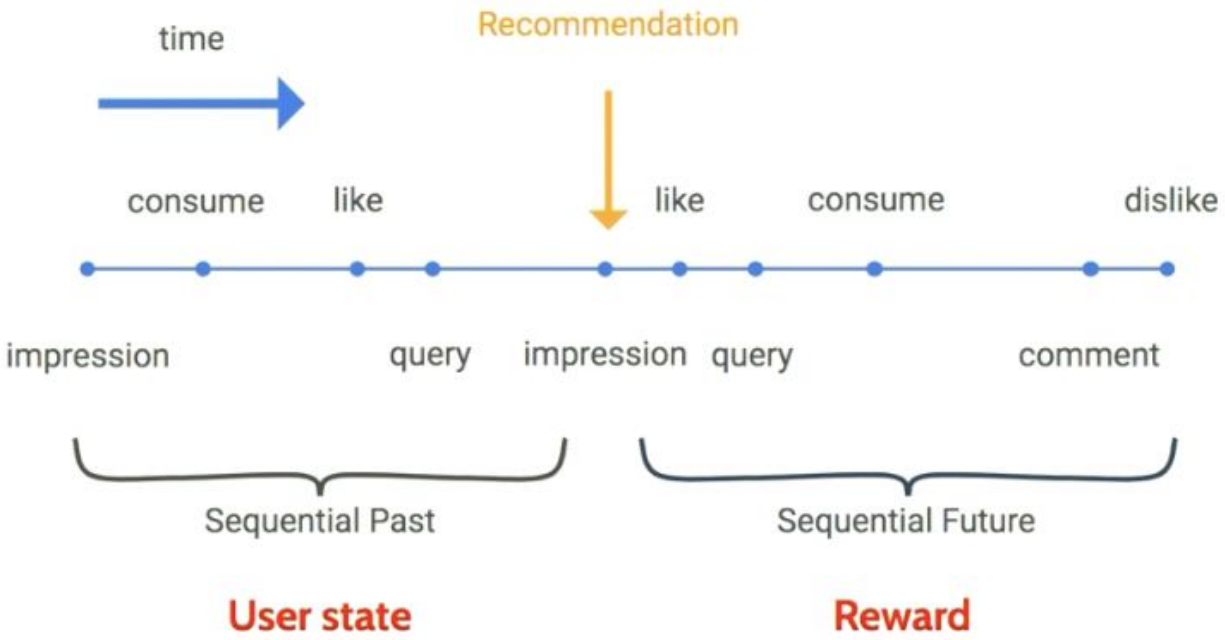
Data Source user Trajectory

Have input on - **Sequential Past** Watched, liked, what was the query



User State

Sequential Past



State Representation (15min)
Sequential Past: to come up with our belief of **User's State**

Sequential Future: to come up with **Reward**

After recommendation, have user feedback (**sequential future**)

- Which videos they **liked** after watching
- Use sequential future to come up with **Reward**

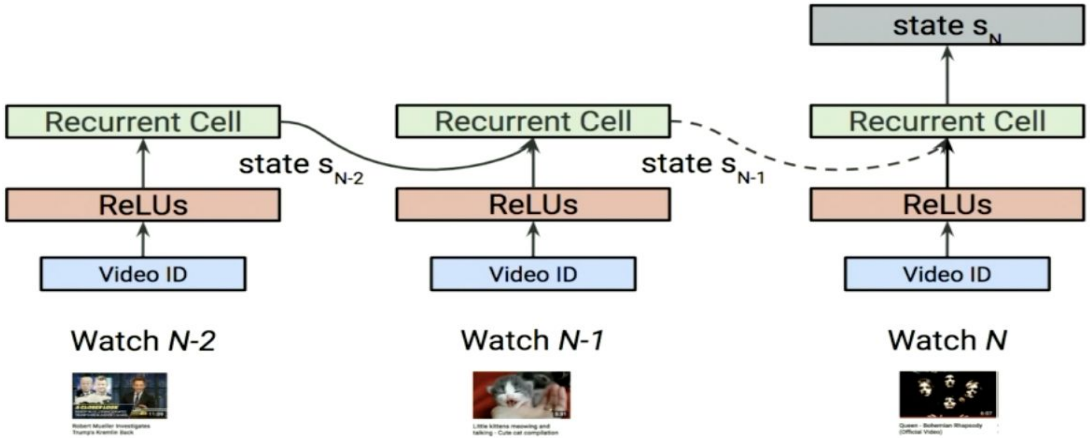
STATE REPRESENTATION

State Representation

Partial observability

Partial Observability (users don't tell us about their interest how happy they are with the recommendation)
Based on user activity - try to figure out what's their interest

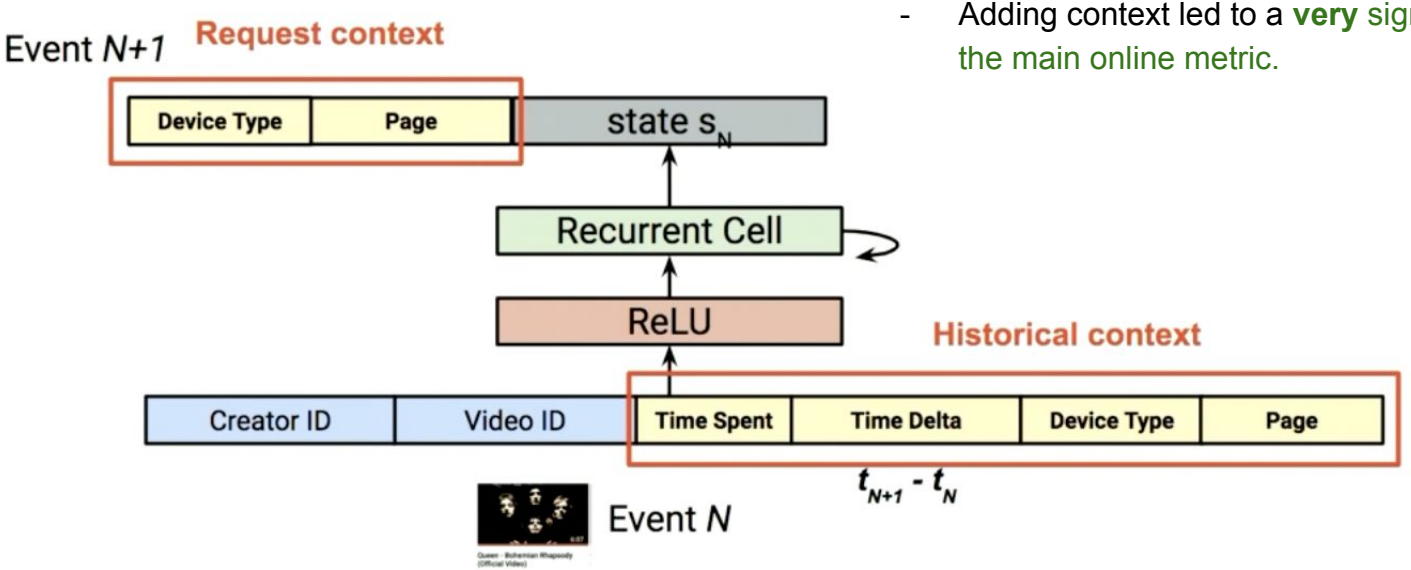
User State Representation through RNNs



Take user watches (right before recommendation) - Send to RNN

RNN will aggregate these events and produce Dense vector which is representation of user state

Adding Context



Beutel, Alex, et al. "Latent Cross: Making Use of Context in Recurrent Recommender Systems." *WSDM* 2018

Historical Representation (context) is very important
Recommend different videos when user is watching on **Desktop** or **Mobile** phone

Live Experiments:
Multiple live experiments improving the state representation using RNNs and

- Adding context led to a **very** significant improvement on the main online metric.

REWARD

Noisy feedback

Reward

Discounted Future Reward

Exponentially discounted future reward,

$$\mathcal{R}_{s_t,a_t} = r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 r(s_{t+2}, a_{t+2}) + \cdots$$

Queen - Love of My Life

4 mins

The Story of Queen: Mercury Rising (FULL...

30 mins

Aerosmith - I Don't Want to Miss a Thing (from You Gott...

4 mins

Aggregate reward, coming after our recommendation -

- Boost (immediate) versus
    - -give action which give long term engagement

To handle noise - aggregate future reward with Exponential Decay

ACTION

Policy-based RL

Goal: Learn a **stochastic policy** $\pi_\theta(a_t|s_t) = \Pr[a_t|s_t;\theta]$ to maximize **cumulative reward**

$$\max_\theta \ \mathbb{E}_{\tau\sim\pi_\theta}[\mathcal{R}(\tau)]$$

Many approaches in RL -> **Policy** based approach:

- Directly tries to maximise the quantity in which we are interested in which is - **long term reward**
- Also, its Stability compared to Value based approach

Stochastic Policy Pi-theta : cast distribution over Action space given a user State

- So that we can maximise this cumulative long term reward

Goal: Learn a **stochastic policy** $\pi_\theta(a_t|s_t) = \Pr[a_t|s_t;\theta]$ to maximize **cumulative reward**

$$\max_\theta \ \mathbb{E}_{\tau\sim\pi_\theta}[\mathcal{R}(\tau)]$$

$$\tau = \{(s_1, a_1), \ldots, (s_T, a_T)\}$$

Trajectory generated by following the policy

$$\mathcal{R}(\tau) = \sum_t r(s_t, a_t)$$

Cumulative reward of the trajectory

- Trajectory tao is generated by following the policy
- Cumulative Reward is total sum of reward for the entire trajectory

Method: Maximize cumulative reward with gradient ascent,

$$\theta \leftarrow \theta + \lambda \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau)]$$

Because we have expressive form of the policy, we can optimise the policy parameter theta by doing gradient ascent

Method: Maximize cumulative reward with gradient ascent,

$$\theta \leftarrow \theta + \lambda \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau)]$$

$$\nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau) \nabla_\theta \log \pi_\theta(\tau)]$$

log-trick

$$= \mathbb{E}_{\tau \sim \pi_\theta}[\sum_t \mathcal{R}(\tau) \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

gradient of weighted log-likelihood

Method: Maximize cumulative reward with gradient ascent,

$$\theta \leftarrow \theta + \lambda \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau)]$$

$$\nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta}[\mathcal{R}(\tau) \nabla_\theta \log \pi_\theta(\tau)]$$

log-trick

$$= \mathbb{E}_{\tau \sim \pi_\theta}[\sum_t \mathcal{R}(\tau) \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

gradient of weighted log-likelihood

Gradient looks like gradient of weighted log-likelihood

- With weight proportional to long term rewards

Learning part - connected to Supervised

- Where you optimise for log-likelihood of observing the next action

RL gives tool to think about

- Exploration, Planning, Changing underlying user state

Large Action Space

Parameterize policy with **softmax over actions**,

$$\pi_\theta(a|s) = \frac{exp(\mathbf{u}_s^\top \mathbf{v}_a)}{\sum_{a'} exp(\mathbf{u}_s^\top \mathbf{v}_{a'})}$$

Sampled softmax at training,
serving is performed by fast nearest neighbor lookup before sampling.

A common choice to parameterise policy is to - do softmax over all the actions

At training - Sampled Softmax:
To deal with large action space of millions

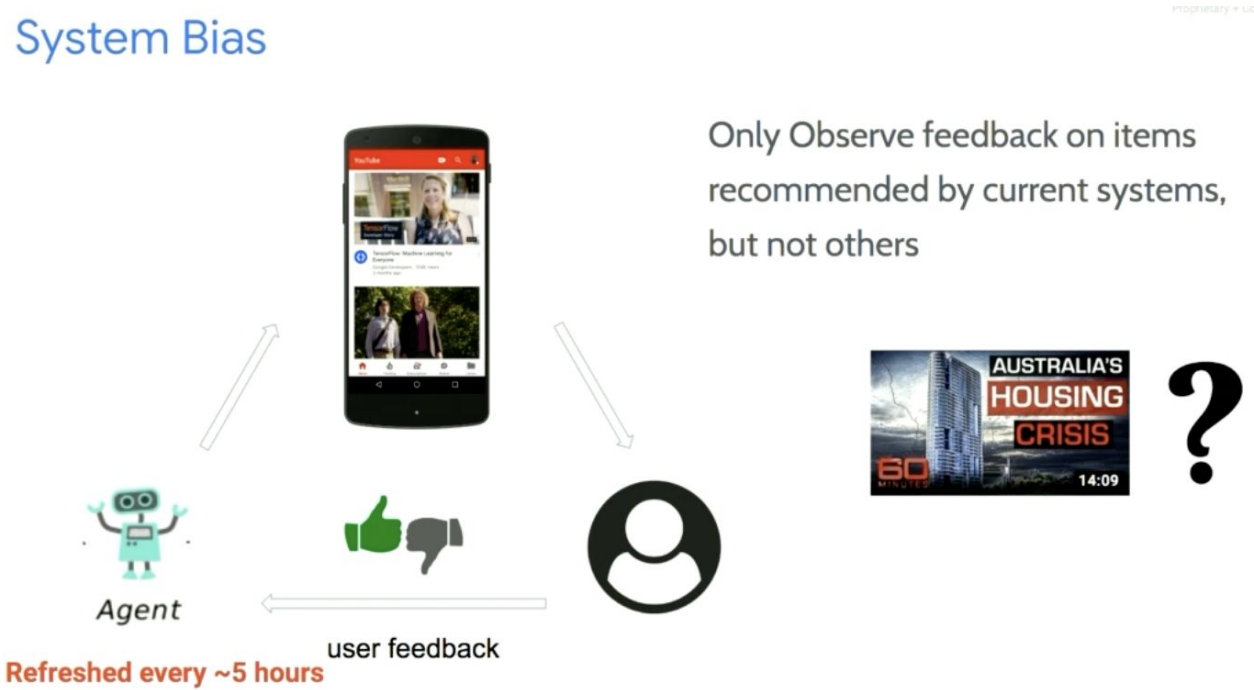
- In learning - did sample softmax
  To Avoid expensive computation of this normalisation factors

At serving time:

- Fast nearest neighbourhood lookup before sampling from subspace of the action space

Off-Policy Learning:

- **Biggest gain - policy learning to address System bias**
- We only have access to data which are off policy



Agent is refreshed every 5 hours

- In 5 hours agent policy will be very different than the target policy we are trying to learn now
- Traffic acquired by other agents will be different policy than the policy we are trying to learn

As a result we have to address system bias which caused by having only access to log data which is generated by different policy

Address System Bias

Logged trajectories

$$\tau_u = \{(s_0, a_0, r_0), (s_1, a_1, r_1), \ldots, (s_{T_u}, a_{T_u}, r_{T_u})\}$$

$$\ldots$$

On-policy update: $$\theta \leftarrow \theta + \lambda \frac{1}{N} \sum_{\tau \sim \pi_\theta} \sum_t \mathcal{R}_{s_t, a_t} \nabla_\theta \log \pi_\theta(a_t | s_t)$$

Off-policy update: $$\theta \leftarrow \theta + \lambda \frac{1}{N} \sum_{\tau \sim \beta_{\theta'}} \frac{\pi_\theta(\tau)}{\beta_{\theta'}(\tau)} \sum_t \mathcal{R}_{s_t, a_t} \nabla_\theta \log \pi_\theta(a_t | s_t)$$

M. Chen*, A. Beutel*, P. Covington*, S. Jain, F. Belletti, E. Chi. Top-K Off-Policy Correction for a REINFORCE Recommender System. ACM International Conference on Web Search and Data Mining, (WSDM), 2019. Google [arxiv]

People have done work (22:22):

- Counter factual learning literature is similar to work domain adaption cases
- Add importance weight - weight a trajectory based on how likely is the trajectory being generated by target policy versus the behaviour policy
- Control bias and variance using the importance weights
- ToDo - for details come to talk on Wed

RESULTS

Live Experiments

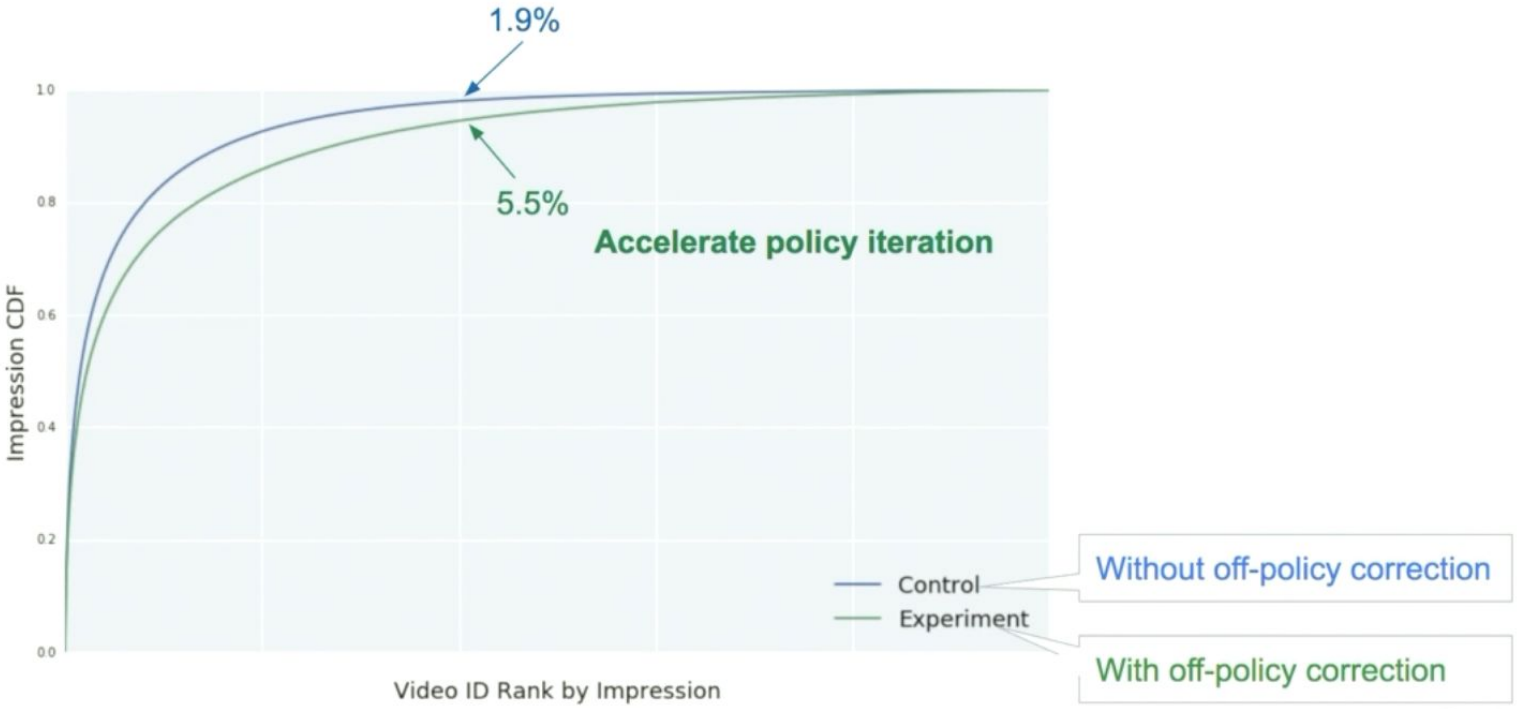
Multiple live experiments improving the state representation using RNNs and adding context lead to a **very** significant improvement on the main online metric.

Live Experiments

	All	Desktop	Mobile	Tablet
Online metric	**0.30%**	0.13%	0.35%	0.17%

Including Future rewards - gave gain in online metrics

Pushing into the Tail

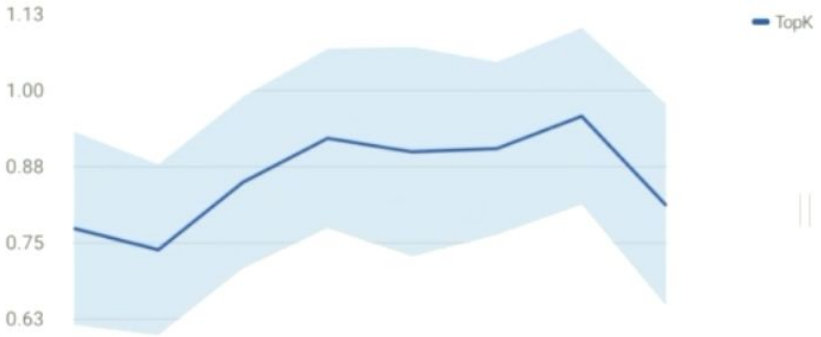


We are like to recommend item which was previously recommended

- but may not be relevant to the user
- **Before off-policy -> much more traffic coming from tail recommendation** Vs before
- Which follows previous system recommending more head videos than tail videos

Live Experiments

	All	Desktop	Mobile	Tablet
Online metric	**0.86%**	0.36%	1.04%	0.68%



0.86 overall metric gain -> largest single launch in YouTube in two years!

Conclusion and Future Works

- Reinforcement Learning Framework for Recommender Systems
- Future Work
  - Better state representation through LRD
  - Better exploration and planning
  - Beyond systems and users: improve Youtube ecosystem.

Better State Representation - we can account for LRD = long range dependencies in User State
Better Exploration and Planning
- So that we lead the Users to different state
- Versus recommending the content familiar to user

We are thinking about users and video
- Include **creators** - have different utility function
- to improve overall YouTube ecosystem instead of just the user utility

Top-K Off-Policy Correction for a REINFORCE Recommender System | AISC

Susan Shu Chang

Formulate the gradient as expectation with log-trick

Math at 45min - log trick

$$\nabla_\theta \mathbb{E}[R(\tau)] = \nabla_\theta \Big(\sum_{t=0}^{|\tau|} \pi_\theta(a_t|s_t) R_t\Big)$$

$$= \sum_{t=0}^{|\tau|} (\nabla_\theta \pi_\theta(a_t|s_t) R_t)$$

Log-trick

$$= \sum_{t=0}^{|\tau|} (\pi_\theta(a_t|s_t) \nabla_\theta \log \pi_\theta(a_t|s_t) R_t)$$

$$= \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \pi_\theta(a_t|s_t)\Big]$$

Log-trick:

$$\pi_\theta(\tau) \nabla_\theta \log \pi_\theta(\tau)$$

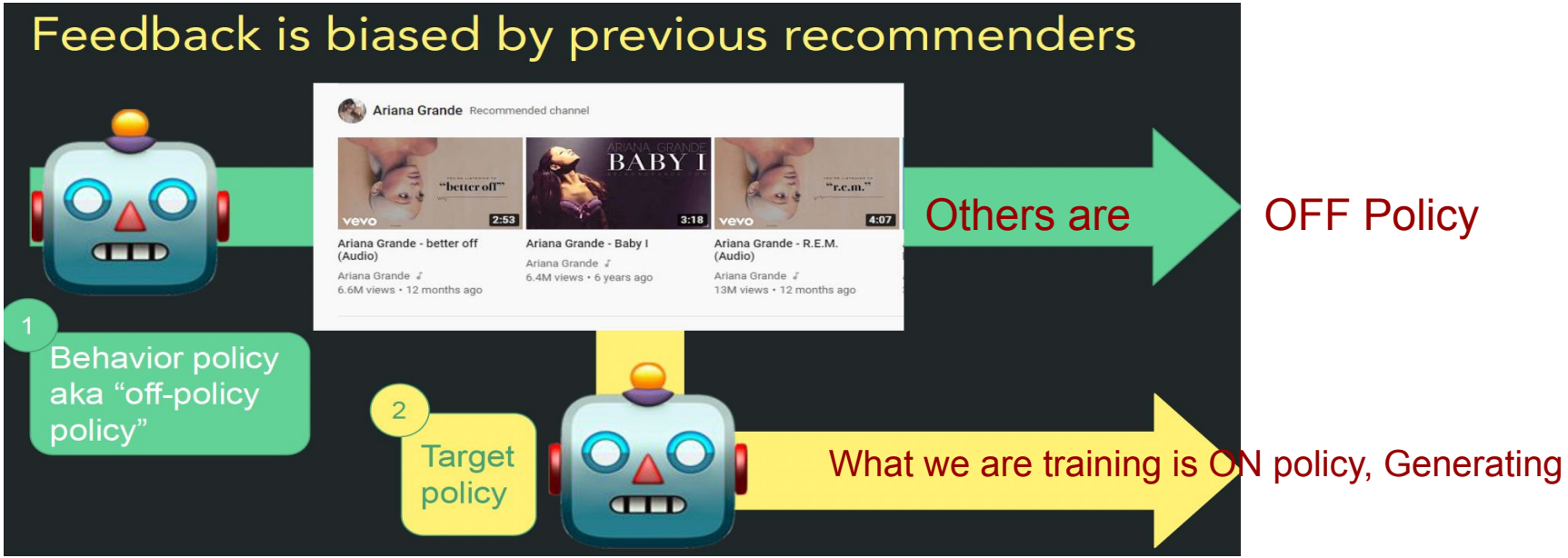
$$= \pi_\theta(\tau) \frac{\nabla_\theta \pi_\theta(\tau)}{\pi_\theta(\tau)}$$

$$= \nabla_\theta \pi_\theta(\tau)$$

Reward:

$$R(\tau) = \sum_{t=0}^{|\tau|} R_t$$

is gradient. Gradient of a function is 1 / that function



Target may not be getting better recommending - because other recommendations are likely to recommend

Policy not be updated instantly which also causes bias

- Feedback could be from other agents in production
- Agent refreshed every ~5 hours
- Learning with off-policy feedback

Correct the biases caused by the "behavior" policy, to improve agent learning

Agent is refreshed every 5 hours.

Part 1
- YouTube recommender system
- REINFORCE algorithm applied to YouTube
- **Off-policy correction**

Part 2
- Results from simulated data and live experiments on YouTube
- Discussion

Off-policy correction downweights non-target policies

- Off-policy gradient update is corrected with importance weight on trajectory based on **how likely the trajectory was generated by behavior policy** vs. target policy
  - Beta-theta-prime - notation

$$\theta \leftarrow \theta + \lambda \sum_{\tau \sim \beta_{\theta'}} \frac{\pi_\theta(\tau)}{\beta_{\theta'}(\tau)} \sum_t R_{s_t,a_t} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

- On-policy update

$$\theta \leftarrow \theta + \lambda \sum_{\tau \sim \pi_\theta} \sum_t R_{s_t,a_t} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Behaviour from other policy is less impactful
- **Downgrade impact of off-policy**

In literature - correction is for top-1 recommend

Policy parametrized with softmax over actions

$$\pi_\theta(a|s) = \frac{\exp(s^\intercal v_a / T)}{\sum_{a' \in \mathcal{A}} \exp(s^\intercal v_{a'} / T)}$$

$$\pi_\theta(a_{t+1}|s_{t+1})$$

→ Then used to update gradient on slide 16

T = temperature turn - smoothness

Part 2 ● **Results from simulated data and live experiments on YouTube** Note: Conducted with simulated data

Not using off-policy correction results in sub-optimal policy

- In simulation they know which video is good
- With off-policy correction - will it recommend
- To validated theoretical understanding

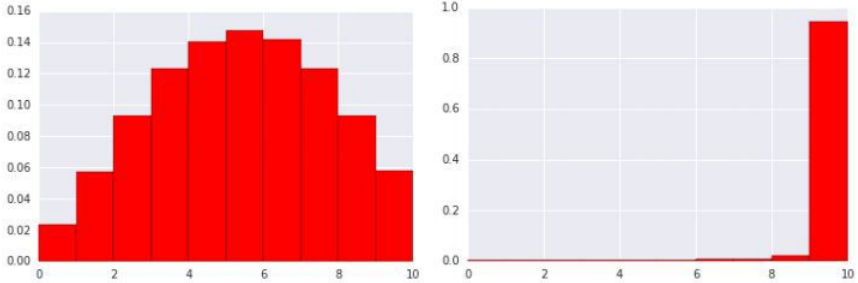


Figure 2: learned policy π_θ when behavior policy β is skewed to favor the actions with least reward, *i.e.*, $\beta(a_i) = \frac{11-i}{55}, \forall i = 1, \cdots, 10$. (left): without off-policy correction; (right): with off-policy correction.

When learn to down weight the off-policy - learns to ignore noise

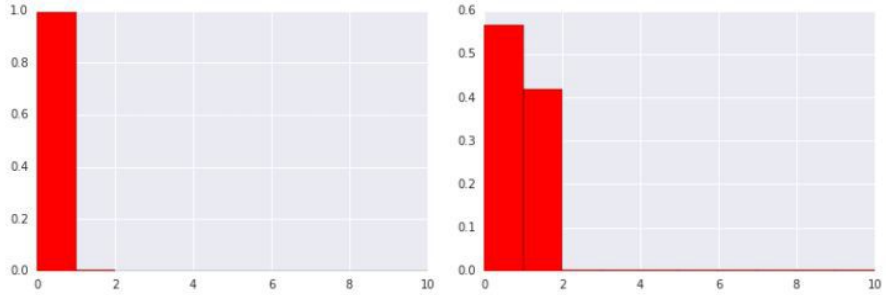


Figure 3: Learned policy π_θ (left): with standard off-policy correction; (right): with top-k correction for top-2 recommendation.

In RL you are taking only one action
In car driving -> left or right or front, stop

- Not together

You can take k actions. K no. of videos

RL Reading Group: Top-K Off-Policy Correction for a REINFORCE Recommender System

Tingting Xu

4. Off-Policy Correction

Distribution mismatch

- the gradient estimation requires sampling trajectories from the policy to update π-theta
- while the trajectories we collected were drawn from a combination of historical policies β.
- A naive policy gradient estimator is no longer unbiased

Importance weighting => unbiased estimation

$$\sum_{\tau \sim \beta} \frac{\pi_\theta(\tau)}{\beta(\tau)} \left[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \left(\pi_\theta(a_t|s_t) \right) \right]$$

- importance weight $$\frac{\pi_\theta(\tau)}{\beta(\tau)} = \frac{\rho(s_0) \prod_{t=0}^{|\tau|} \mathbf{P}(s_{t+1}|s_t, a_t) \pi(a_t|s_t)}{\rho(s_0) \prod_{t=0}^{|\tau|} \mathbf{P}(s_{t+1}|s_t, a_t) \beta(a_t|s_t)} = \prod_{t=0}^{|\tau|} \frac{\pi(a_t|s_t)}{\beta(a_t|s_t)}$$
- unbiased estimate if the trajectories are collected with actions sampled according to β

Huge variance of the estimator

- chained products leads to very low or high values of the importance weights

Reduce Variance of Off-Policy Gradient

- unbiased estimation w/ high variance $\sum_{\tau\sim\beta}\frac{\pi_\theta(\tau)}{\beta(\tau)}\left[\sum_{t=0}^{|\tau|}R_t\nabla_\theta\log\left(\pi_\theta(a_t|s_t)\right)\right]$
  - importance weight $\frac{\pi_\theta(\tau)}{\beta(\tau)}=\frac{\rho(s_0)\prod_{t=0}^{|\tau|}\mathbf{P}(s_{t+1}|s_t,a_t)\pi\left(a_t|s_t\right)}{\rho(s_0)\prod_{t=0}^{|\tau|}\mathbf{P}(s_{t+1}|s_t,a_t)\beta\left(a_t|s_t\right)}=\prod_{t=0}^{|\tau|}\frac{\pi\left(a_t|s_t\right)}{\beta\left(a_t|s_t\right)}$
  - unbiased estimate if the trajectories are collected with actions sampled according to β

To reduce the variance of each gradient term corresponding to a time t, approximate the importance weights

- first ignore the terms after time t in the chained products
- further take a first-order approximation

$$\prod_{t'=0}^{|\tau|}\frac{\pi\left(a_{t'}|s_{t'}\right)}{\beta\left(a_{t'}|s_{t'}\right)}\approx\prod_{t'=0}^{t}\frac{\pi\left(a_{t'}|s_{t'}\right)}{\beta\left(a_{t'}|s_{t'}\right)}=\frac{P_{\pi_\theta}\left(s_t\right)}{P_\beta\left(s_t\right)}\frac{\pi\left(a_t|s_t\right)}{\beta\left(a_t|s_t\right)}\approx\frac{\pi\left(a_t|s_t\right)}{\beta\left(a_t|s_t\right)}.$$

a biased estimator of the policy gradient with lower variance

$$\sum_{\tau\sim\beta}\left[\sum_{t=0}^{|\tau|}\frac{\pi_\theta\left(a_t|s_t\right)}{\beta\left(a_t|s_t\right)}R_t\nabla_\theta\log\pi_\theta(a_t|s_t)\right]$$

Reduce Variance of Off-Policy Gradient

A biased estimator of the policy gradient with lower variance

$$\sum_{\tau \sim \beta} \left[\sum_{t=0}^{|\tau|} \frac{\pi_\theta(a_t|s_t)}{\beta(a_t|s_t)} R_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right]$$

Achiam et al. [1] prove that the impact of this first-order approximation on the total reward of the learned policy is bounded in magnitude

$$O\left(E_{s \sim d^\beta}\left[D_{TV}(\pi|\beta)[s]\right]\right)$$

- where DTV is the total variation between π(·|s) and β(·|s)
- d^β is the discounted future state distribution under β.

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained policy optimization. arXiv preprint arXiv:1705.10528

4.1 Parametrize the policy π_θ

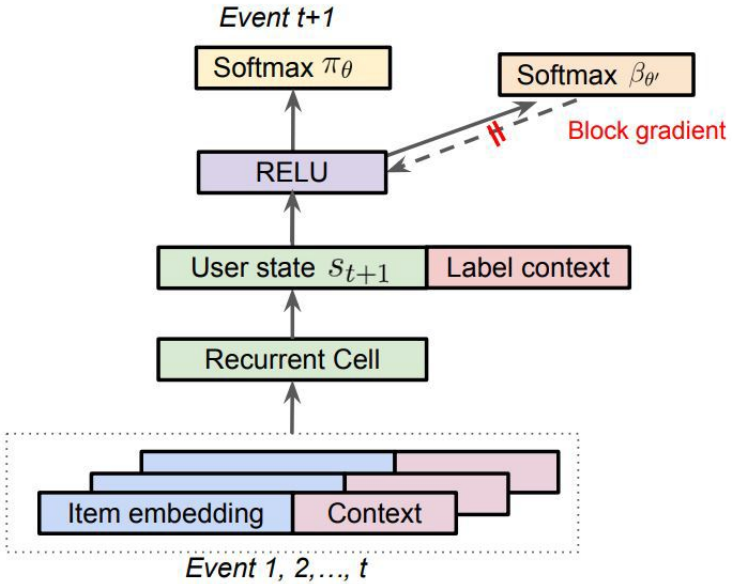


Figure 1: A diagram shows the parametrisation of the policy π_θ as well as the behavior policy $\beta_{\theta'}$.

The state transition is modeled with a recurrent neural network

Yes

$$\mathbf{s}_{t+1} = \mathbf{z}_t \odot \tanh(\mathbf{s}_t) + \mathbf{i}_t \odot \tanh(\mathbf{W}_a \mathbf{u}_{a_t})$$
$$\mathbf{z}_t = \sigma(\mathbf{U}_z \mathbf{s}_t + \mathbf{W}_z \mathbf{u}_{a_t} + \mathbf{b}_z)$$
$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{s}_t + \mathbf{W}_i \mathbf{u}_{a_t} + \mathbf{b}_i)$$

where $\mathbf{z}_t, \mathbf{i}_t \in \mathbb{R}^n$ are the update and input gate respectively.

Conditioning on a user state $\mathbf{s}$, the policy $\pi_\theta(a|\mathbf{s})$ is then modeled with a simple softmax,

$$\pi_\theta(a|\mathbf{s}) = \frac{exp(\mathbf{s}^\top \mathbf{v}_a / T)}{\sum_{a' \in \mathcal{A}} exp(\mathbf{s}^\top \mathbf{v}_{a'} / T)} \quad (5)$$

- where $v_a \in R^n$ is another embedding for each action a in the action space A
- T is a temperature that is normally set to 1.

4.2 Estimating the behavior policy β

- re-use the user states generated from the RNN model
  - model policy β with another softmax layer.
- prevent the behavior head from intefering with the user state of the main policy
  - block its gradient from flowing back into the RNN

4.3 Top-K Off-Policy Correction

- Challenge
  - Policy $\Pi_\Theta(A|s)$: action A is to select a set of k items.
  - Recommend a page of k items to users at a time.
- Objective $\max_\Theta \mathbb{E}_{\tau\sim\Pi_\theta}\left[\sum_t r(s_t, A_t)\right]$
  - expectation over trajectories $\tau = (s_0, A_0, s_1, \cdots)$
  - where $s_0 \sim \rho_0$, $A_t \sim \Pi(\cdot|s_t)$, $s_{t+1} \sim P(\cdot|s_t, a_t)$
- Assume
  - the expected reward of a set of non-repetitive items equal to the sum of the expected reward of each item in the set.
  - generate the set action A by independently sampling each item a according to the softmax policy π_θ and then de-duplicate.

$$\Pi_\Theta(A'|s) = \prod_{a\in A'} \pi_\theta(a|s)$$

- Adapt the REINFORCE algorithm to the set recommendation

$$\sum_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \alpha_\theta(a_t|s_t)\right]$$

where $\alpha_\theta(a|s) = 1-(1-\pi_\theta(a|s))^K$ is the probability that an item a appears in the final non-repetitive set A.

4.3 Top-K Off-Policy Correction

a biased estimator of the policy gradient with lower variance

$$\sum_{\tau\sim\beta}\left[\sum_{t=0}^{|\tau|}\frac{\alpha_\theta(a_t|s_t)}{\beta(a_t|s_t)}R_t\nabla_\theta\log\alpha_\theta(a_t|s_t)\right] \tag{6}$$

$$=\sum_{\tau\sim\beta}\left[\sum_{t=0}^{|\tau|}\frac{\pi_\theta(a_t|s_t)}{\beta(a_t|s_t)}\frac{\partial\alpha(a_t|s_t)}{\partial\pi(a_t|s_t)}R_t\nabla_\theta\log\pi_\theta(a_t|s_t)\right].$$

where $\alpha_\theta(a|s) = 1-(1-\pi_\theta(a|s))^K$

- top-K policy adds an additional multiplier to the original off-policy correction factor

$$\lambda_K(s_t,a_t)=\frac{\partial\alpha(a_t|s_t)}{\partial\pi(a_t|s_t)}=K(1-\pi_\theta(a_t|s_t))^{K-1} \tag{7}$$

As $\pi_\theta(a|s)\to 0, \lambda_K(s,a)\to K$. increases the policy update by a factor of K; (add likelihood for small-mass items)

As $\pi_\theta(a|s)\to 1, \lambda_K(s,a)\to 0$. zeros out the policy update. (no longer increase likelihood if the desirable item already has reasonable mass, i.e., likely to appear in the top-K)

=> allow other items of interest to take up some mass in the softmax policy.

4.4 Variance Reduction Techniques

- Importance weight: $\omega(s,a) = \frac{\pi(a|s)}{\beta(a|s)}$
- Large importance weight due to:
  - large deviation of the new policy π from the behavior policy
    - the new policy explores regions that are less explored by the behavior policy.
    - $\pi(a|s) \gg \beta(a|s)$
  - large variance in the β estimate

Weight Capping. The first approach we take is to simply cap the weight [8] as

$$\bar{\omega}_c(s,a) = \min\left(\frac{\pi(a|s)}{\beta(a|s)}, c\right). \tag{8}$$

Smaller value of c reduces variance in the gradient estimate, but introduces larger bias.

Normalized Importance Sampling (NIS). Second technique we employed is to introduce a ratio control variate, where we use classical weight normalization [32] defined by:

$$\bar{\omega}_n(s,a) = \frac{\omega(s,a)}{\sum_{(s',a')\sim\beta} \omega(s',a')}.$$

As $\mathbb{E}_\beta[\omega(s,a)] = 1$, the normalizing constant is equal to n, the batch size, in expectation. As n increases, the effect of NIS is equivalent to tuning down the learning rate.

Yes

Trusted Region Policy Optimization (TRPO) TRPO [36]

- prevents the new policy π from deviating from the behavior policy
- by adding a regularization that penalizes the KL divergence of these two policies.
- It achieves similar effect as the weight capping.

5 Exploration:

Boltzmann exploration

- get the benefit of exploratory data without negatively impacting user experience

consider using a stochastic policy where recommendations are sampled from π-theta rather than taking the K items with the highest probability.

- efficient approximate nearest neighbor-based systems to look up the top M items in the softmax feed the logits of these M items into a smaller softmax to normalize the probabilities and sample from this distribution.
- $M \gg K$, we can still retrieve most of the probability mass, limit the risk of bad recommendations, and enable computationally efficient sampling.

In practice

- we further balance exploration and exploitation by returning the top K' most probable items and sample $K - K'$ items from the remaining $M - K'$ items.

Contributions

REINFORCE Recommender:

- We scale a REINFORCE policy gradient-based approach to learn a neural recommendation policy in an extremely large action space.

Off-Policy Candidate Generation:

- We apply off-policy correction to learn from logged feedback, collected from an ensemble of prior model policies.
- We incorporate a learned neural model of the behavior policies to correct data biases.

Top-K Off-Policy Correction:

- We offer a novel top-K off policy correction to account for the fact that our recommender outputs multiple items at a time.

Benefits in Live Experiments:

- We demonstrate in live experiments, which was rarely done in existing RL literature, the value of these approaches to improve user long term satisfaction.

THANK YOU